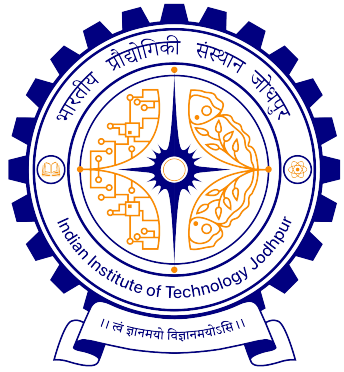
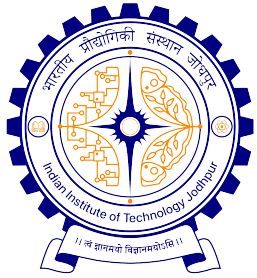




Introduction to Computer Science (CSL1010)

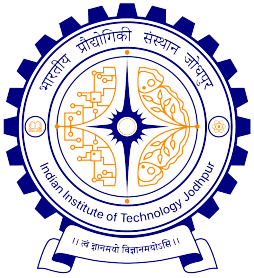
C Basics– Function L9

Suchetana Chakraborty
Department of Computer Science, IIT Jodhpur
Spring, 2026



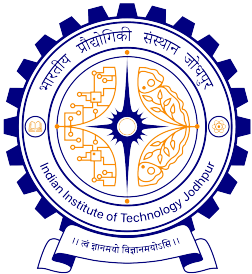
Function

- A **program segment** that carries out some **specific, well-defined task**
- Example
 - A function to add two numbers
 - A function to find the largest of n numbers
- A function will carry out its intended task whenever it is **called** or **invoked**
 - Can be called multiple times



Function

- Every C program consists of one or more functions
- One of these functions must be called **main**
- Execution of the program always begins by carrying out the instructions in **main**
- Functions call other functions as instructions



Function Control Flow

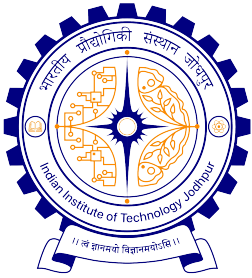
```
void print_banner ()  
{  
    printf("*****\n");  
}
```

```
void main ()  
{  
    . . .  
    print_banner ();  
    . . .  
    print_banner ();  
}
```

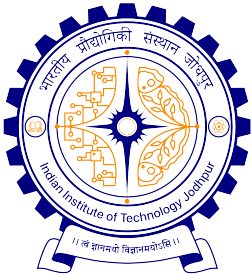
```
int main ()  
{  
    print_banner ();  
  
    print_banner ();  
}
```

```
print_banner {  
}
```

```
print_banner {  
}
```



- Calling function (**caller**) may pass information to the called function (**callee**) as parameters/arguments
 - For example, the numbers to add
- The callee may return **a single value** to the caller
 - Some functions may not return anything



Calling function (Caller)

```
void main()  
{ float cent, fahr;  
  scanf("%f",&cent);  
  fahr = cent2fahr(cent);  
  printf("%fC = %fF\n",  
    cent, fahr);  
}
```

Calling/Invoking the cent2fahr function

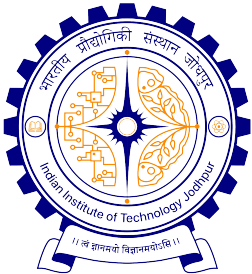
Called function (Callee)

```
float cent2fahr(float data)  
{  
  float result;  
  result = data*9/5 + 32;  
  return result;  
}
```

parameter

Parameter passed

Returning value



How it runs

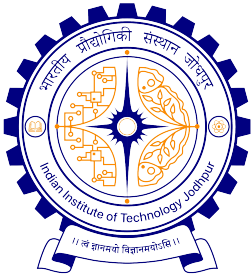
```
float cent2fahr(float data)
{
    float result;
    printf("data = %f\n", data);
    result = data*9/5 + 32;
    return result;
    printf("result = %f\n", result);
}

void main()
{ float cent, fahr;
  scanf("%f",&cent);
  printf("Input is %f\n", cent);
  fahr = cent2fahr(cent);
  printf("%fC = %fF\n", cent, fahr);
}
```

Output

```
$ ./a.out
32
Input is 32.000000
data = 32.000000
32.000000C = 89.599998F

$./a.out
-45.6
Input is -45.599998
data = -45.599998
-45.599998C = -50.079998F
$
```



Another Example

```
int factorial
(int m)
{
    int i, temp=1;
    for (i=1; i<=m;
        i++)
        temp = temp *
        i;
    return (temp);
}
```

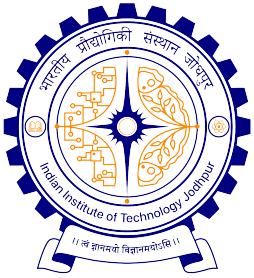
```
void main()
{
    int    n;
    for    (n=1; n<=10; n++)
        printf ("%d! = %d \n",
                n, factorial (n)
        );
}
```

Output

1! = 1

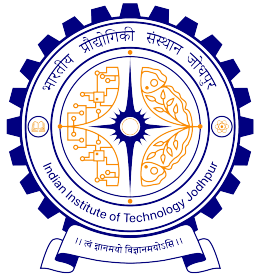
2! = 2

3! = 6 upto 10!



Why Functions?

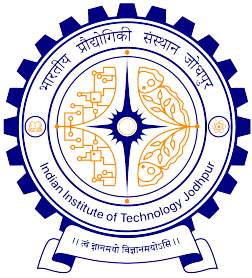
- **Modularity:** Allows one to develop a program in a modular fashion
 - Divide-and-conquer approach
 - Construct a program from small pieces or components
- **Reusability:** Use existing functions as building blocks for new programs
- **Abstraction:** hide internal details (library functions)



Defining a Function

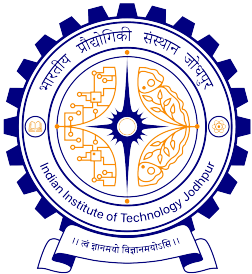
- A function definition has **two** parts:
 - The first line, called **header**
 - The **body** of the function

```
return-value-type  function-name ( parameter-list )  
{  
    declarations and statements  
}
```



Defining a Function

- The first line contains the return-value-type, the function name, and optionally a set of comma-separated arguments enclosed in parentheses
 - Each argument has an associated type declaration
 - The arguments are called **formal arguments** or **formal parameters**
- The body of the function is actually a block of statement that defines the action to be taken by the function



Return-value type

Formal parameters

`int gcd (int A, int B)`

`{`

`int temp;`

`while ((B % A) != 0) {`

`temp = B % A;`

`B = A;`

`A = temp;`

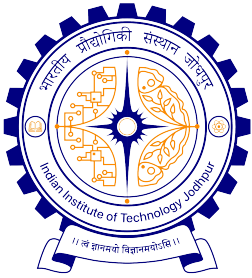
`}`

`return (A);`

`}`

BODY

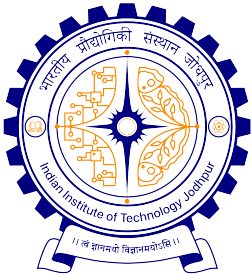
**Value
returned**



Return value

- A function can return a value
 - Using **return** statement
- Like all values in C, a function return value has a type
- The return value can be assigned to a variable (mind the type) in the caller

```
int x, y, z;  
scanf("%d%d", &x, &y);  
z = gcd(x,y);  
printf("GCD of %d and %d is %d\n", x, y, z);
```



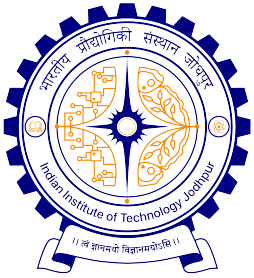
Function Not Returning Any Value

- Example: A function which prints if a number is divisible by 7 or not

```
void div7 (int n)
{
    if ((n % 7) == 0)
        printf ("%d is divisible by 7", n);
    else
        printf ("%d is not divisible by 7", n);
    return;
}
```

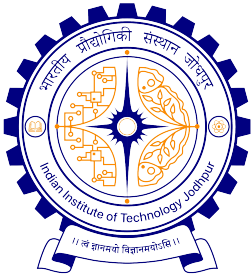
Return type is void

Optional



return statement

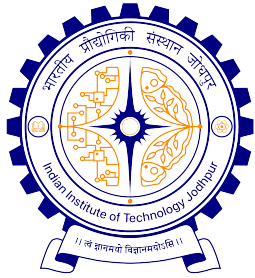
- In a value-returning function (result type is **not** void), **return** does two distinct things
 - **specify the value returned** by the execution of the function
 - **terminate that execution of the callee** and transfer control back to the caller
- **A function can only return one value**
 - The value can be any expression **matching the return type**
 - but it **might contain more than one return statement**.
- In a void function
 - return is optional at the end of the function body.
 - return may also be used to terminate execution of the function explicitly.
 - No return value should appear following return.



```
void compute_and_print_itax ()
```

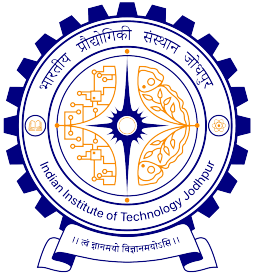
```
{  
    float income;  
    scanf ("%f", &income);  
    if (income < 50000) {  
        printf ("Income tax = Nil\n");  
        return;  
    }  
    if (income < 60000) {  
        printf ("Income tax = %f\n", 0.1*(income-50000));  
        return;  
    }  
    if (income < 150000) {  
        printf ("Income tax = %f\n", 0.2*(income-60000)+1000);  
        return ;  
    }  
    printf ("Income tax = %f\n", 0.3*(income-150000)+19000);  
}
```

Terminate function
execution before
reaching the end



Calling a function

- Called by specifying the function name and parameters in an instruction in the calling function
- When a function is called from some other function, the corresponding arguments in the function call are called **actual arguments or actual parameters**
 - The function call **must include a matching actual parameter for each formal parameter**
 - **Position of an actual parameters in the parameter list in the call must match** the position of the corresponding formal parameter in the function definition
 - **The formal and actual arguments must match in their data types**



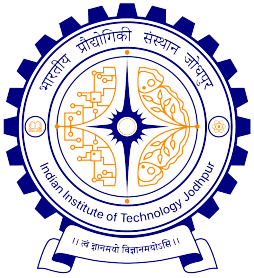
Example

Formal parameters

```
void main ()
{
    double x, y, z;
    char op;
    . . .
    z = operate (x, y, op);
    . . .
}
```

```
double operate(double x, double y, char op)
{
    switch (op) {
        case '+' : return x+y+0.5 ;
        case '~'  : if (x>y)
                    return x-y + 0.5;
                    return y-x+0.5;
        case 'x' : return x*y + 0.5;
        default : return -1;
    }
}
```

Actual parameters



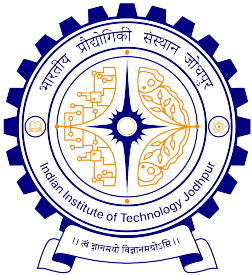
Parameter Passing

- When the function is executed, the **value of the actual parameter is copied to the formal parameter**

```
void main ()  
{  
    . . .  
    double circum;  
    . . .  
    area1 = area(circum/2.0) ;  
    . . .  
}
```

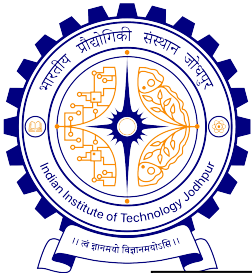
parameter passing

```
double area (double r)  
{  
    return (3.14*r*r) ;  
}
```



Another Example

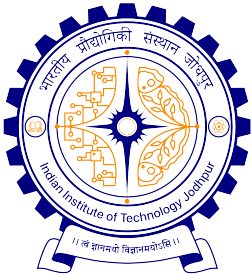
```
/* Compute the GCD of four numbers */  
void main()  
{  
    int  n1, n2, n3, n4, result;  
    scanf ("%d %d %d %d", &n1, &n2, &n3, &n4);  
    result  =  gcd ( gcd (n1, n2), gcd (n3, n4) );  
    printf ("The GCD of %d, %d, %d and %d is %d \n", n1,  
n2, n3, n4, result);  
}
```



Another Example

```
void main()
{
    int numb, flag, j=3;
    scanf("%d",&numb);
    while (j <=numb)
    {
        flag = prime(j);
        if (flag==0)
            printf("%d is
prime\n",j);
        j++;
    }
}
```

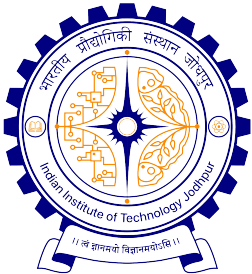
```
int prime(int x)
{
    int i, test;
    i=2, test =0;
    while ((i <= sqrt(x)) && (test ==0))
    {
        if (x%i==0) test = 1;
        i++;
    }
    return test;
}
```



Tracking the flow of control

```
void main()
{
    int numb, flag, j=3;
    scanf("%d",&numb);
    printf("numb = %d \n",numb);
    while (j <= numb)
    {   printf("Main, j = %d\n",j);
        flag = prime(j);
        printf("Main, flag = %d\n",flag);
        if (flag == 0)
            printf("%d is prime\n",j);j++;
    }
}
```

```
int prime(int x)
{
    int i, test;
    i = 2;  test = 0;
    printf("In function, x = %d \n",x);
    while ((i <= sqrt(x)) && (test == 0))
    {   if (x%i == 0) test = 1;
        i++;
    }
    printf("Returning, test = %d \n",test);
    return test;
}
```



The output

5

numb = 5

Main, j = 3

In function, x = 3

Returning, test = 0

Main, flag = 0

3 is prime

Main, j = 4

In function, x = 4

Returning, test = 1

Main, flag = 1

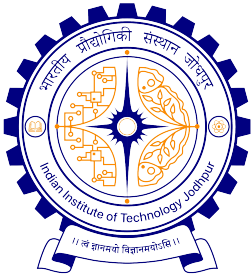
Main, j = 5

In function, x = 5

Returning, test = 0

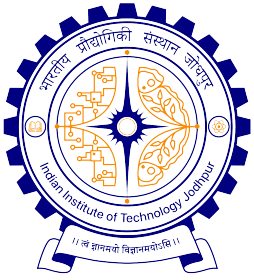
Main, flag = 0

5 is prime



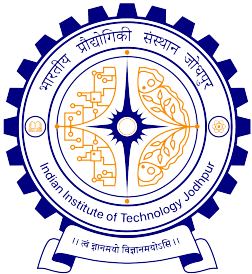
Points to note

- The identifiers used as formal parameters are “local”.
 - Not recognized outside the function
 - Names of formal and actual arguments may differ
- A value-returning function is called by including it in an expression
 - A function with return type T ($\neq$ void) can be used anywhere an expression of type T can be used



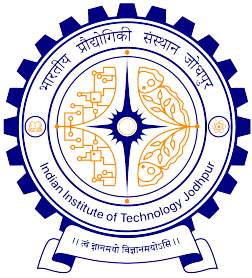
Returning Control to Caller

- Returning control back to the caller
 - If nothing returned
 - `return;`
 - or, until reaches the last right brace ending the function body
 - If something returned
 - `return expression;`



Function Prototypes

- Usually, a function is **defined before it is called**
 - `main()` is the last function in the program written
 - Easy for the compiler to identify function definitions in a single scan through the file
- However, many programmers prefer a top-down approach, where the functions are written after `main()`
 - Must be some way to tell the compiler
 - Function prototypes are used for this purpose
 - **Only needed if function definition comes after use**

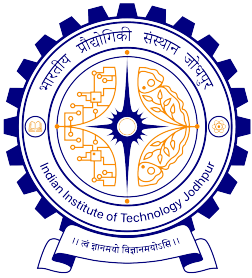


Function Prototypes

- Function prototypes are usually written at the beginning of a program, ahead of any functions (including `main()`)
- Prototypes can specify parameter names or just types (more common)
- Examples:

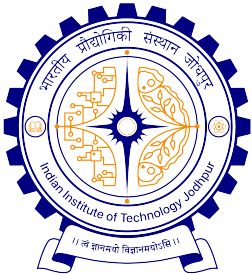
```
int gcd (int , int );  
void div7 (int number);
```

- Note the semicolon at the end of the line.
- The parameter name, if specified, can be anything; but it is a good practice to use the same names as in the function definition



Some more points

- A function cannot be defined within another function
 - All function definitions must be disjoint
- Nested function calls are allowed
 - A calls B, B calls C, C calls D, etc.
 - The function called last will be the first to return
- A function can also call itself, either directly or in a cycle
 - A calls B, B calls C, C calls back A.
 - Called **recursive call** or **recursion**



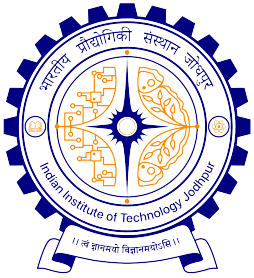
Example: **main** calls **ncr**, **ncr** calls **fact**

```
int ncr (int n, int r);
int fact (int n);

void main()
{
    int i, m, n, sum=0;
    scanf ("%d %d", &m, &n);
    for (i=1; i<=m; i+=2)
        sum = sum + ncr (n,
i);
    printf ("Result: %d \n",
sum);
}
```

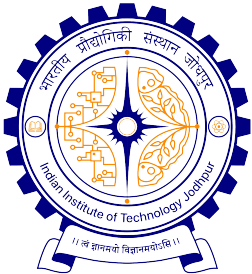
```
int ncr (int n, int r)
{
    return (fact(n) / fact(r) /
fact(n-r));
}

int fact (int n)
{
    int i, temp=1;
    for (i=1; i<=n; i++)
        temp *= i;
    return (temp);
}
```



Local variables

- A function can define its own local variables
- The locals have meaning only within the function
 - Each execution of the function uses a new set of locals
 - Local variables cease to exist when the function returns
- Parameters are also local

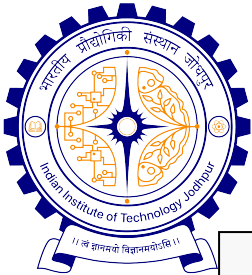


Local variables

```
/* Find the area of a circle with diameter d */  
double circle_area (double d)  
{  
    double radius, area;  
    radius = d/2.0;  
    area = 3.14*radius*radius;  
    return (area);  
}
```

parameter

local variables



Revisiting nCr

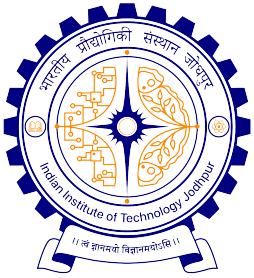
```
int fact(int x)
{ int i,fact=1;
  for(i=2; i<=x; ++i) fact=fact*i;
  return fact;
}
```

```
int ncr(int x,int y)
{
    int p,q,r;
    p=fact(x);
    q=fact(y);
    r = fact(x-y);
    return p/(q*r);
}
```

```
void main()
{
    int n, r;
    scanf("%d%d",&n,&r);
    printf("n=%d, r=%d,
    nCr=%d\n",n, r, ncr(n,r));
}
```

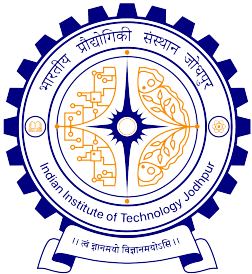
The variable `x` in function `fact` and `x` in function `ncr` are different.

The values computed from the arguments at the point of call are copied on to the corresponding parameters of the called function before it starts execution.



Scope of a variable

- Part of the program from which the value of the variable can be used (seen)
- Scope of a variable - Within the block in which the variable is defined
 - Block = group of statements enclosed within { }
- **Local variable** – scope is usually the function in which it is defined
 - So two local variables of two functions can have the same name, but they are different variables
- **Global variables** – declared outside all functions (even main)
 - scope is entire program by default, but can be hidden in a block if local variable of same name defined



```
#include <stdio.h>
```

```
int A = 1;
```

```
void main()
```

```
{
```

```
    myProc();
```

```
    printf ( "A = %d\n", A);
```

```
}
```

```
void myProc()
```

```
{
```

```
    int A = 2;
```

```
    if ( A==2 )
```

```
    {
```

```
        int A = 3;
```

```
        printf ( "A = %d\n", A);
```

```
    }
```

```
    printf ( "A = %d\n", A);
```

```
}
```

Variable Scope

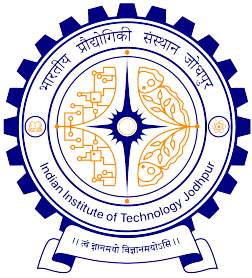
Global variable

A = 1

Hides the global A

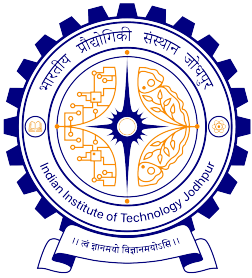
A = 3

A = 2



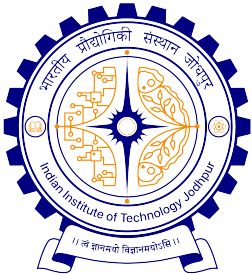
Parameter Passing

- Used when invoking functions
- Call by value
 - Passes the value of the argument to the function
 - Execution of the function does not change the actual parameters
 - All changes to a parameter done inside the function are done on a copy of the actual parameter
 - The copy is removed when the function returns to the caller
 - The value of the actual parameter in the caller is not affected
 - Avoids accidental changes



Parameter Passing

- Call by reference
 - Passes the **address** to the original argument.
 - Execution of the function may affect the original
 - Not directly supported in C except for arrays



Parameter passing & return: 1

```
void main()
{
    int  a = 10, b;
    printf  ("Initially a = %d\n", a);
    b = change (a);
    printf  ("a = %d, b = %d\n", a, b);
}

int  change  (int x)
{
    printf  ("Before x = %d\n", x);
    x = x / 2;
    printf  ("After x = %d\n", x);
    return (x);
}
```

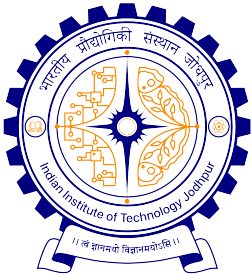
Output

Initially a = 10

Before x = 10

After x = 5

a = 10, b = 5



Parameter passing & return: 2

```
void main()
{
    int  x = 10, b;
    printf ("M: Initially x = %d\n", x);
    b = change (x);
    printf ("M: x = %d, b = %d\n", x, b);
}

int  change  (int x)
{
    printf ("F: Before x = %d\n", x);
    x = x / 2;
    printf ("F: After x = %d\n", x);
    return (x);
}
```

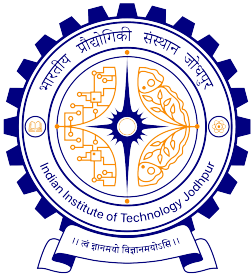
Output

M: Initially x = 10

F: Before x = 10

F: After x = 5

M: x = 10, b = 5



Parameter passing & return: 3

```
void main()
{
    int  x = 10, b;
    printf ("M: Initially x = %d\n", x);
    x = change (x);
    printf ("M: x = %d, b = %d\n", x, x);
}

int  change  (int x)
{
    printf ("F: Before x = %d\n", x);
    x = x / 2;
    printf ("F: After x = %d\n", x);
    return (x);
}
```

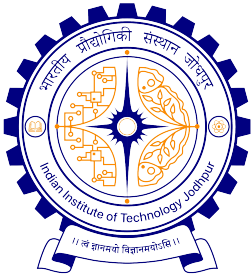
Output

M: Initially x = 10

F: Before x = 10

F: After x = 5

M: x = 5, b = 5



Parameter passing & return: 4

```
void main()
{
    int  x=10, y=5;
    printf ("M1:  x = %d, y = %d\n", x, y);
    interchange (x, y);
    printf ("M2:  x = %d, y = %d\n", x, y);
}

void interchange (int x, int y)
{ int temp;
  printf ("F1:  x = %d, y = %d\n", x, y);
  temp= x; x = y; y = temp;
  printf ("F2:  x = %d, y = %d\n", x, y);
}
```

Output

M1: x = 10, y = 5

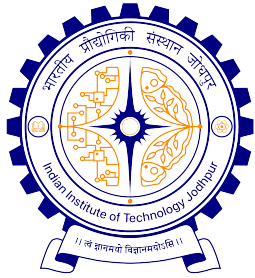
F1: x = 10, y = 5

F2: x = 5, y = 10

M2: x = 10, y = 5

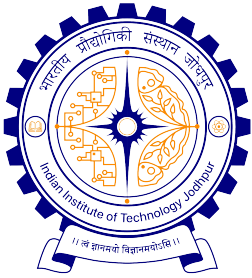
How do we write an
interchange function?

**Hint: the change needs to
persist beyond function scope!**



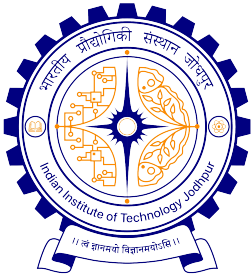
Passing Arrays to Function

- Array element can be passed to functions as ordinary arguments
 - `IsFactor (x[i], x[0]);`
 - `sin (x[5]);`



Passing Entire Array to a Function

- An **array name** can be used as an argument to a function
 - Permits the entire array to be passed to the function
 - The way it is passed differs from that for ordinary variables
- **Rules:**
 - The array name must appear by itself as argument, **without brackets or subscripts**
 - The corresponding **formal argument is written in the same manner**
 - Declared by writing the array name with a pair of empty brackets



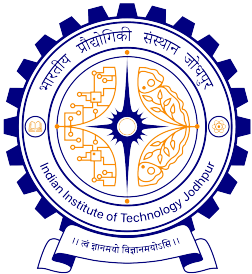
Whole Array as Parameters

```
const int ASIZE = 5;
float average (int B[ ])
{
    int i, total=0;
    for (i=0; i<ASIZE; i++)
        total = total + B[i];
    return ((float) total / (float) ASIZE);
}
```

Only Array Name/address passed.
[] mentioned to indicate that
is an array.

```
void main ( ) {
    int x[ASIZE] ; float x_avg;
    x = {10, 20, 30, 40, 50};
    x_avg = average (x) ;
}
```

Called only with actual array name



Contd.

We don't need to write the array size. It works with arrays of any size.

it is not compulsory as such
but if the data of array size is required
we need to pass it explicitly

```
void main()
{
    int n;
    float list[100], avg;
    :
    avg = average (n, list);
    :
}

float average (int a, float x[])
{
    :
    sum = sum + x[i];
}
```

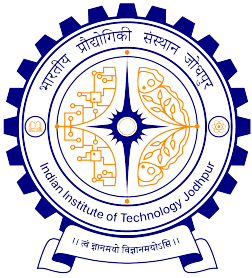


Arrays used as Output Parameters

```
void VectorSum (int a[ ], int b[ ], int vsum[ ], int length) {
    int i;
    for (i=0; i<length; i=i+1)
        vsum[i] = a[i] + b[i] ;
}

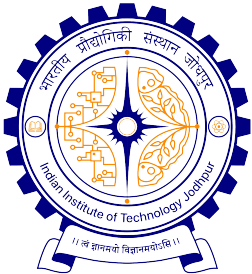
void PrintVector (int a[ ], int length) {
    int i;
    for (i=0; i<length; i++) printf ("%d ", a[i]);
}

void main () {
    int x[3] = {1,2,3}, y[3] = {4,5,6}, z[3];
    VectorSum (x, y, z, 3) ;
    PrintVector (z, 3) ;
}
```



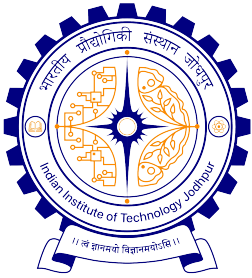
The Actual Mechanism

- When an array is passed to a function, the values of the array elements are **not passed** to the function
 - The array name is interpreted as the **address** of the first array element
 - The formal argument therefore becomes a **pointer** to the first array element
 - **When an array element is accessed inside the function, the address is calculated using the formula stated before**
 - **Changes made inside the function are thus also reflected in the calling program (unlike passing a variable)**



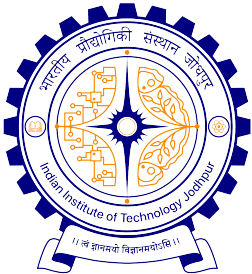
Contd.

- Passing parameters in this way is called **call-by-reference**
- Normally parameters are passed in C using **call-by-value**
- Basically what it means?
 - If a function changes the values of array elements, then these changes will be made to the original array that is passed to the function
 - This does not apply when an individual element is passed on as argument



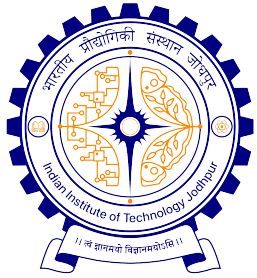
Header Files

- Header files:
 - Contain function declarations / prototypes for library functions.
 - `<stdlib.h>` , `<math.h>` , etc.
 - Load with: `#include <filename>`
 - Example: `#include <math.h>`
 - The function definitions of library functions are in the actual libraries (e.g., math library).
- We can also create custom header files:
 - Create file(s) with function prototypes / declarations.
 - Save as `filename.h` (say).
 - Load in other files with `#include "filename.h"`



C Preprocessor

- Statements starting with # are handled by the C preprocessor
- May be done by the compiler or by a separate program
- Preprocessing is done before the actual compilation process begins
- The C preprocessor is basically a text substitution tool
- For instance, #include command is replaced by the contents of the specified header file
- Such commands are called **preprocessor directives**



#define: Macro definition

- Preprocessor directive in the following form:

#define string1 string2

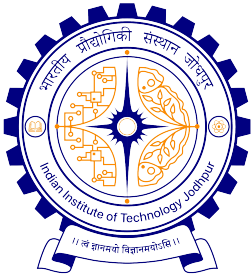
- Replaces string1 by string2 wherever it occurs before compilation. For example,

#define PI 3.1415926

```
#define PI 3.1415926
int main()
{
    float r = 4.0, area;
    area = PI * r * r;
    return 0;
}
```

macro pre-processing

```
int main()
{
    float r = 4.0, area;
    area = 3.1415926 * r * r;
    return 0;
}
```



#define with arguments

- #define statement may be used with arguments.

- Example: `#define sqr(x) x*x`

- How will macro substitution be carried out?

`r = sqr(a) + sqr(30);` $\square$ `r = a*a + 30*30;`

`r = sqr(a+b);` $\square$ `r = a+b*a+b;` **//WRONG**

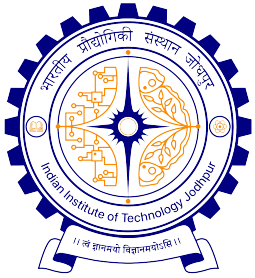
- The macro should better be written as:

`#define sqr(x) (x)*(x)`

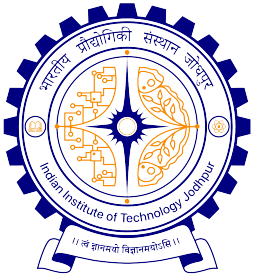
`r = sqr(a+b);` $\square$ `r = (a+b)*(a+b);`

`r = c / sqr(a+b);` $\square$ `r = c / (a+b)*(a+b);` **//Observe when it can fail!**

- **Macros are not functions. They are literally substituted without evaluation.**



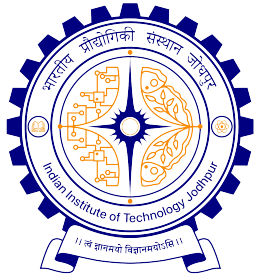
Recursion



Recursion

- A process by which a function calls itself repeatedly.
- Either directly.
 - F calls F.
- Or cyclically in a chain.
 - F calls G, G calls H, and H calls F.
- Used for repetitive computations in which each action is stated in terms of a previous result.

$$\text{fact}(n) = n * \text{fact}(n-1)$$



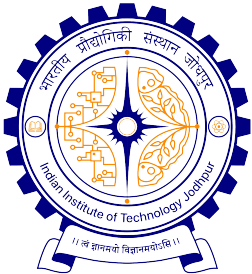
Contd.

- For a problem to be written in recursive form, two conditions are to be satisfied:
 - It should be **possible to express the problem in recursive form**
 - Solution of the problem in terms of solution of the same problem on smaller sized data
 - The problem statement **must include a stopping condition**

$$\begin{aligned} \text{fact}(n) &= 1, && \text{if } n = 0 \\ &= n * \text{fact}(n-1), && \text{if } n > 0 \end{aligned}$$

← **Stopping condition**

← **Recursive definition**



Examples:

- **Factorial:**

$$n! = n \times (n-1) !$$

$$\text{fact}(0) = 1$$

$$\text{fact}(n) = n * \text{fact}(n-1), \text{ if } n > 0$$

- **GCD:**

$$\text{gcd}(m, n) = \text{if } m > n \text{ gcd}(m-n, n) \text{ else gcd}(m, n-m)$$

$$\text{gcd}(m, m) = m$$

$$\text{gcd}(m, n) = \text{gcd}(m \% n, n), \text{ if } m > n$$

$$\text{gcd}(m, n) = \text{gcd}(n, n \% m), \text{ if } m < n$$

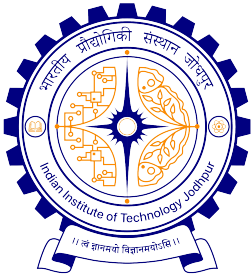
- **Fibonacci series (1,1,2,3,5,8,13,21,...)**

$$\text{fib}(0) = 1$$

$$\text{fib}(1) = 1$$

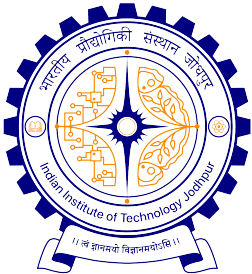
$$\text{fib}(n) = \text{fib}(n-1) + \text{fib}(n-2), \text{ if } n > 1$$

$$n_i = n_{i-1} + n_{i-2}$$



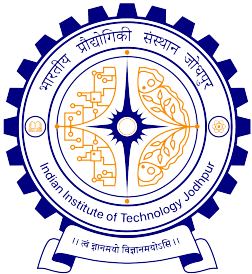
Factorial

```
long int fact (int n)
{
    if (n == 1)
        return (1) ;
    else
        return (n * fact(n-1)) ;
}
```

Factorial Execution

```
long int fact (int n)
{
    if (n == 1) return (1);
    else return (n * fact(n-1));
}
```

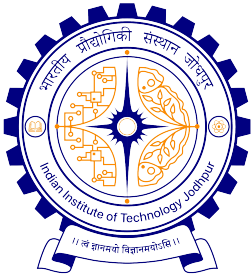


Factorial Execution

fact(4)



```
long int fact (int n)
{
    if (n == 1) return (1);
    else return (n * fact(n-1));
}
```



Factorial Execution

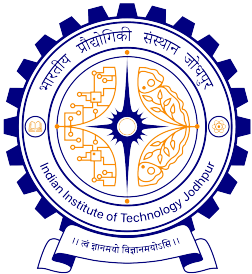
fact(4)



if (**4** == 1) return (1);
else return (**4** * **fact(3)**);



```
long int fact (int n)
{
    if (n == 1) return (1);
    else return (n * fact(n-1));
}
```



Factorial Execution

fact(4)



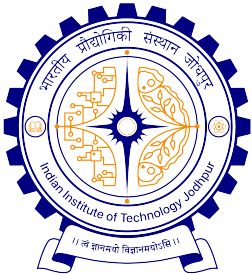
if (**4** == 1) return (1);
else return (**4** * **fact(3)**);



if (**3** == 1) return (1);
else return (**3** * **fact(2)**);



```
long int fact (int n)
{
    if (n == 1) return (1);
    else return (n * fact(n-1));
}
```



Factorial Execution

fact(4)



if (**4** == 1) return (1);
else return (**4** * **fact(3)**);

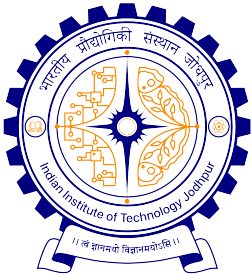


if (**3** == 1) return (1);
else return (**3** * **fact(2)**);



if (**2** == 1) return (1);
else return (**2** * **fact(1)**);

```
long int fact (int n)
{
    if (n == 1) return (1);
    else return (n * fact(n-1));
}
```



Factorial Execution

fact(4)



if (**4** == 1) return (1);
else return (**4** * **fact(3)**);



if (**3** == 1) return (1);
else return (**3** * **fact(2)**);

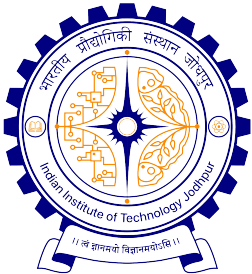


if (**2** == 1) return (1);
else return (**2** * **fact(1)**);



if (**1** == 1) **return (1)**;

```
long int fact (int n)
{
    if (n == 1) return (1);
    else return (n * fact(n-1));
}
```



Factorial Execution

fact(4)



if (**4** == 1) return (1);
else return (**4** * **fact(3)**);



if (**3** == 1) return (1);
else return (**3** * **fact(2)**);



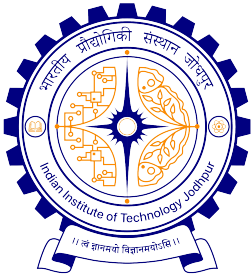
if (**2** == 1) return (1);
else return (**2** * **fact(1)**);

1



if (**1** == 1) **return (1)**;

```
long int fact (int n)
{
    if (n == 1) return (1);
    else return (n * fact(n-1));
}
```



Factorial Execution

fact(4)



if (**4** == 1) return (1);
else return (**4** * **fact(3)**);



if (**3** == 1) return (1);
else return (**3** * **fact(2)**);



if (**2** == 1) return (1);
else return (**2** * **fact(1)**);

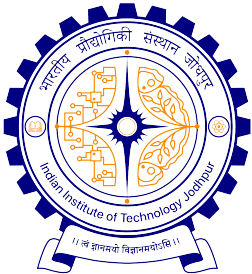


if (**1** == 1) **return (1)**;

2

1

```
long int fact (int n)
{
    if (n == 1) return (1);
    else return (n * fact(n-1));
}
```

Factorial Execution

fact(4)



if (**4** == 1) return (1);
else return (**4** * **fact(3)**);



if (**3** == 1) return (1);
else return (**3** * **fact(2)**);



if (**2** == 1) return (1);
else return (**2** * **fact(1)**);

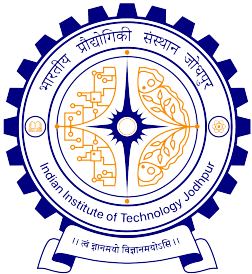


if (**1** == 1) **return (1)**;

2

1

```
long int fact (int n)
{
    if (n == 1) return (1);
    else return (n * fact(n-1));
}
```



Factorial Execution

fact(4)

if (**4** == 1) return (1);
else return (**4** * **fact(3)**);



if (**3** == 1) return (1);
else return (**3** * **fact(2)**);

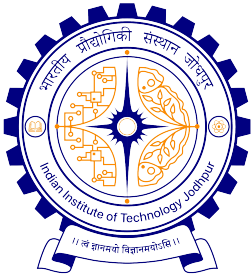


if (**2** == 1) return (1);
else return (**2** * **fact(1)**);

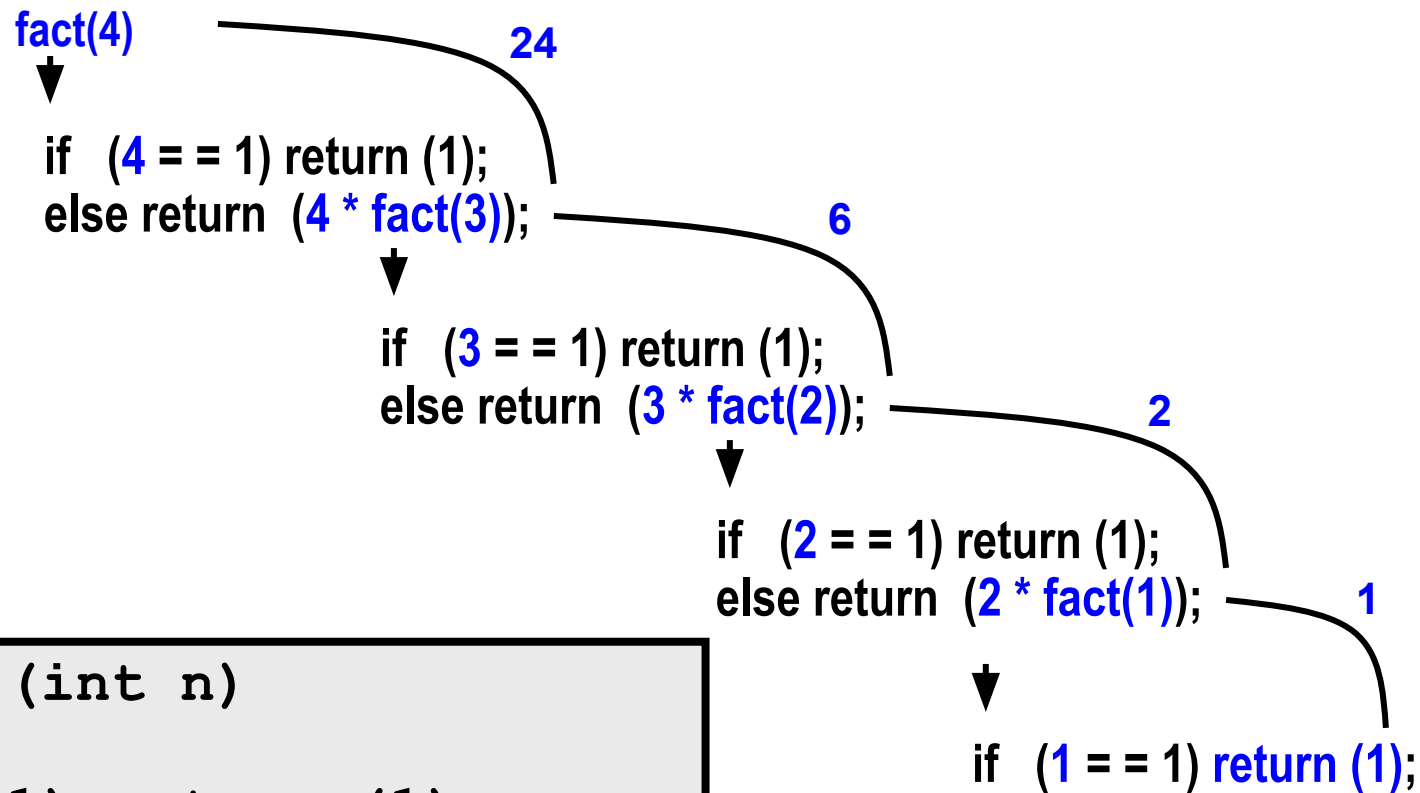


if (**1** == 1) return (**1**);

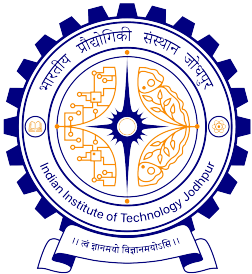
```
long int fact (int n)
{
    if (n == 1) return (1);
    else return (n * fact(n-1));
}
```



Factorial Execution



```
long int fact (int n)
{
    if (n == 1) return (1);
    else return (n * fact(n-1));
}
```



Look at the variable addresses (a slightly different program) !

```
void main()
{
    int x,y;
    scanf("%d",&x);
    y = fact(x);
    printf ("M: x= %d, y = %d\n", x,y);
}

int fact(int data)
{ int val = 1;
  printf("F: data = %d, &data = %u \n
    &val = %u\n", data, &data, &val);
  if (data>1) val = data*fact(data-1);
  return val;
}
```

Output

4

F: data = 4, &data = 3221224528

&val = 3221224516

F: data = 3, &data = 3221224480

&val = 3221224468

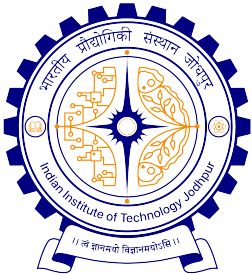
F: data = 2, &data = 3221224432

&val = 3221224420

F: data = 1, &data = 3221224384

&val = 3221224372

M: x= 4, y = 24

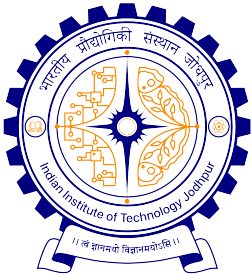


Fibonacci Numbers

Fibonacci recurrence:

**$\text{fib}(n) = 1$ if $n = 0$ or 1 ;
 $= \text{fib}(n - 2) + \text{fib}(n - 1)$
 otherwise;**

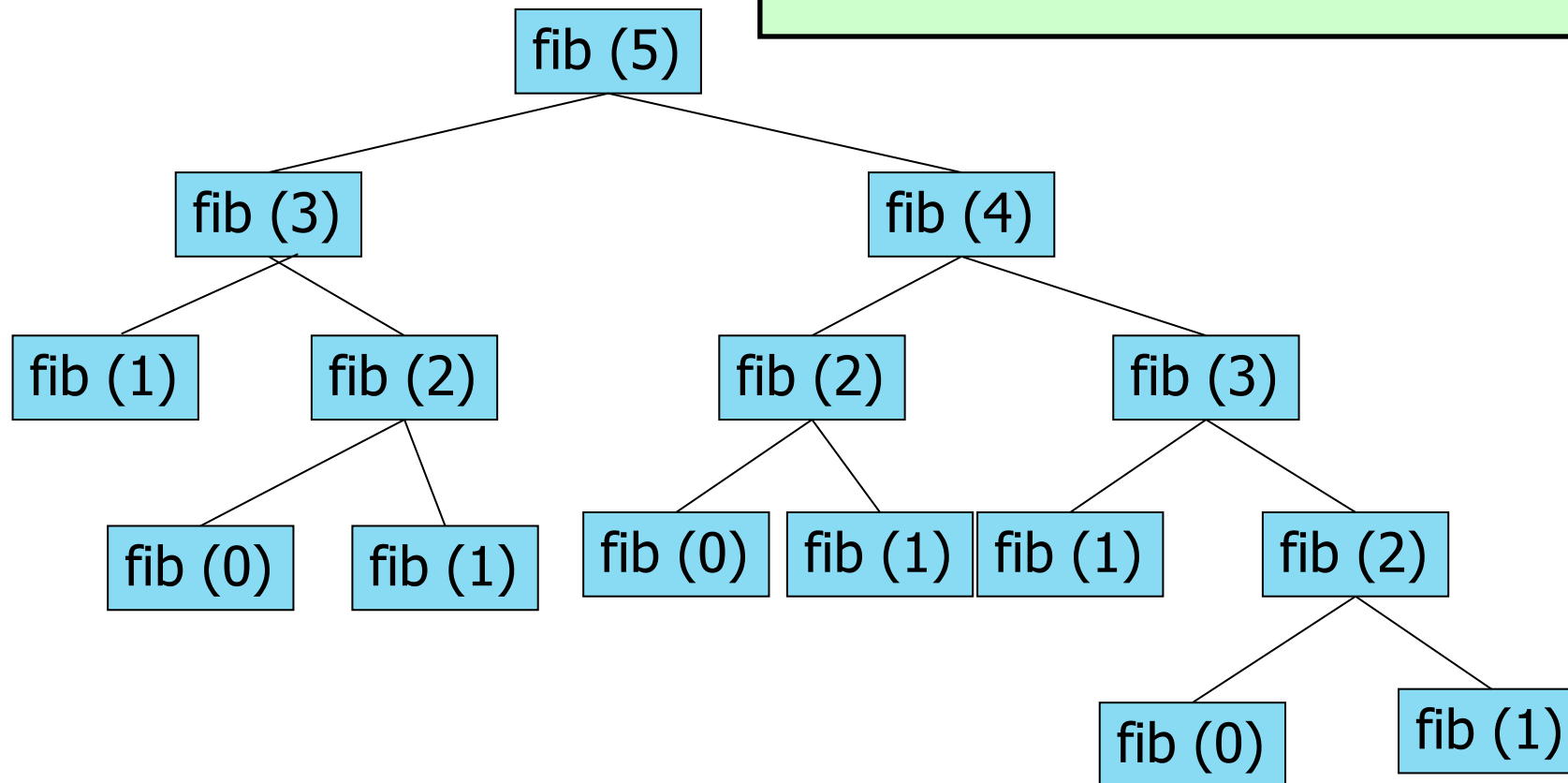
```
int fib (int n) {  
    if (n == 0 or n == 1)  
        return 1;    // [BASE]  
    return fib(n-2) + fib(n-1) ; // [Recursive]  
}
```

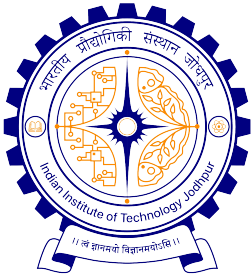


```
int fib (int n)  {  
    if (n == 0 || n == 1)  
        return 1;  
    return fib(n-2) + fib(n-1) ;  
}
```

Fibonacci recurrence:

$\text{fib}(n) = 1$ if $n = 0$ or 1 ;
 $= \text{fib}(n - 2) + \text{fib}(n - 1)$
otherwise;

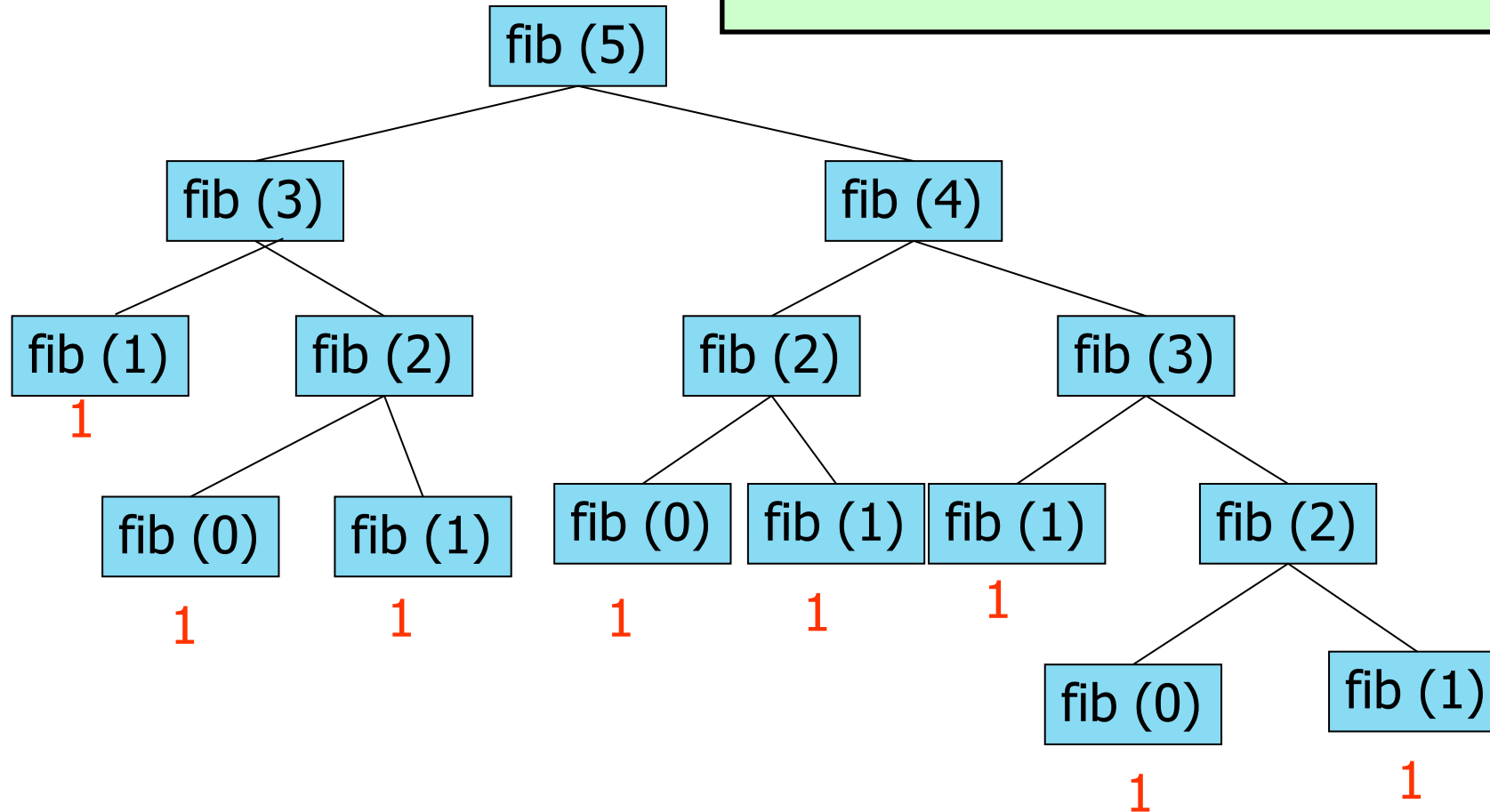


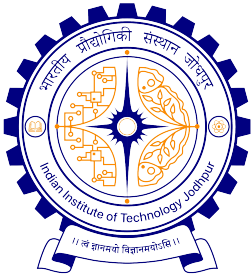


```
int fib (int n)  {  
    if (n == 0 || n == 1)  
        return 1;  
    return fib(n-2) + fib(n-1) ;  
}
```

Fibonacci recurrence:

$\text{fib}(n) = 1$ if $n = 0$ or 1 ;
 $= \text{fib}(n - 2) + \text{fib}(n - 1)$
otherwise;

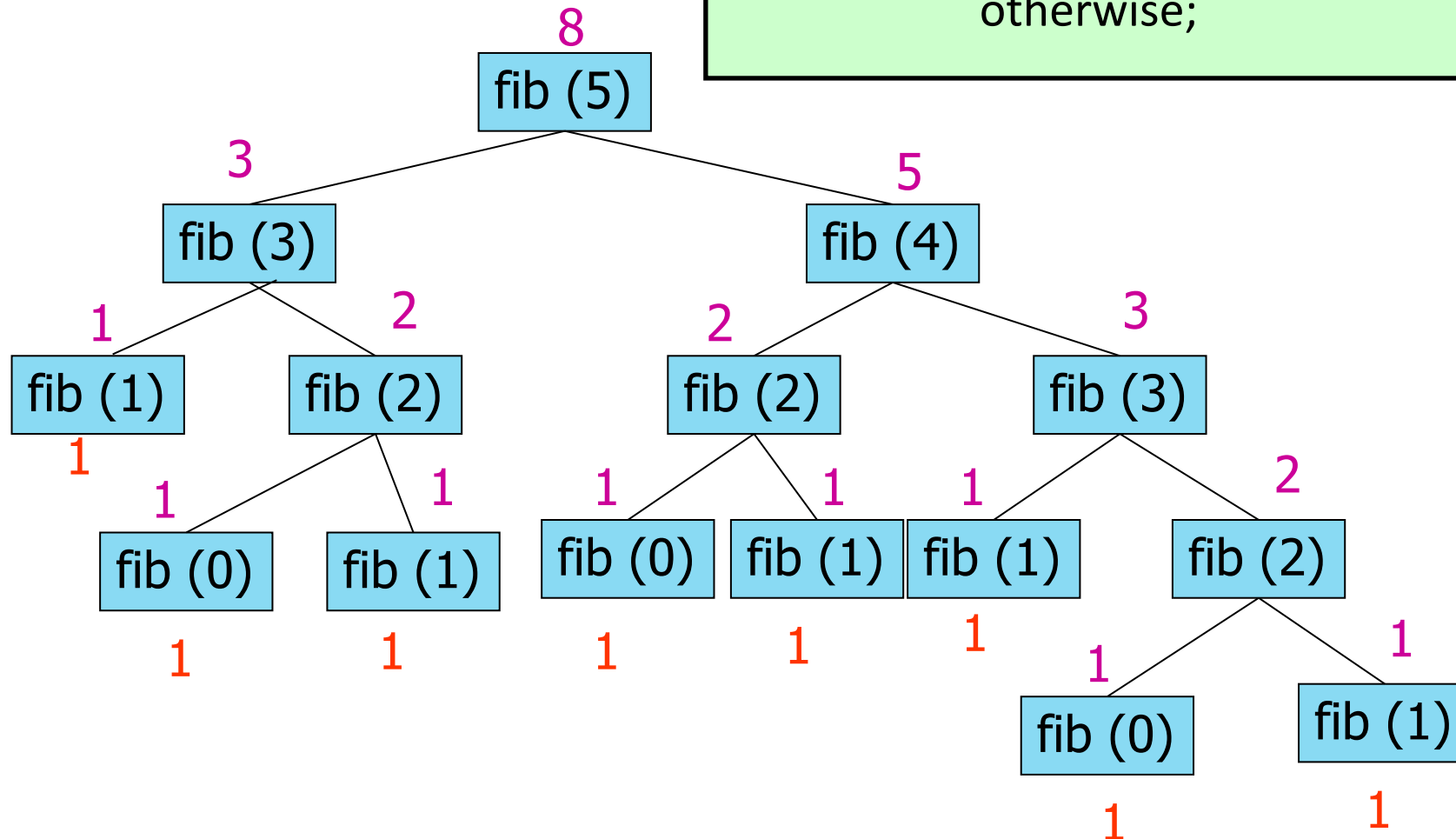


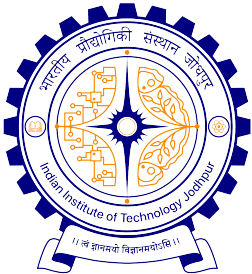


```
int fib (int n)  {  
    if (n==0 || n==1)  
        return 1;  
    return fib(n-2) + fib(n-1) ;  
}
```

Fibonacci recurrence:

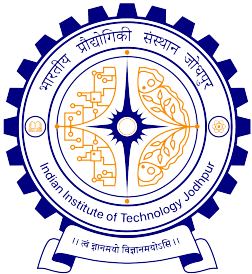
$\text{fib}(n) = 1$ if $n = 0$ or 1 ;
 $= \text{fib}(n - 2) + \text{fib}(n - 1)$
otherwise;



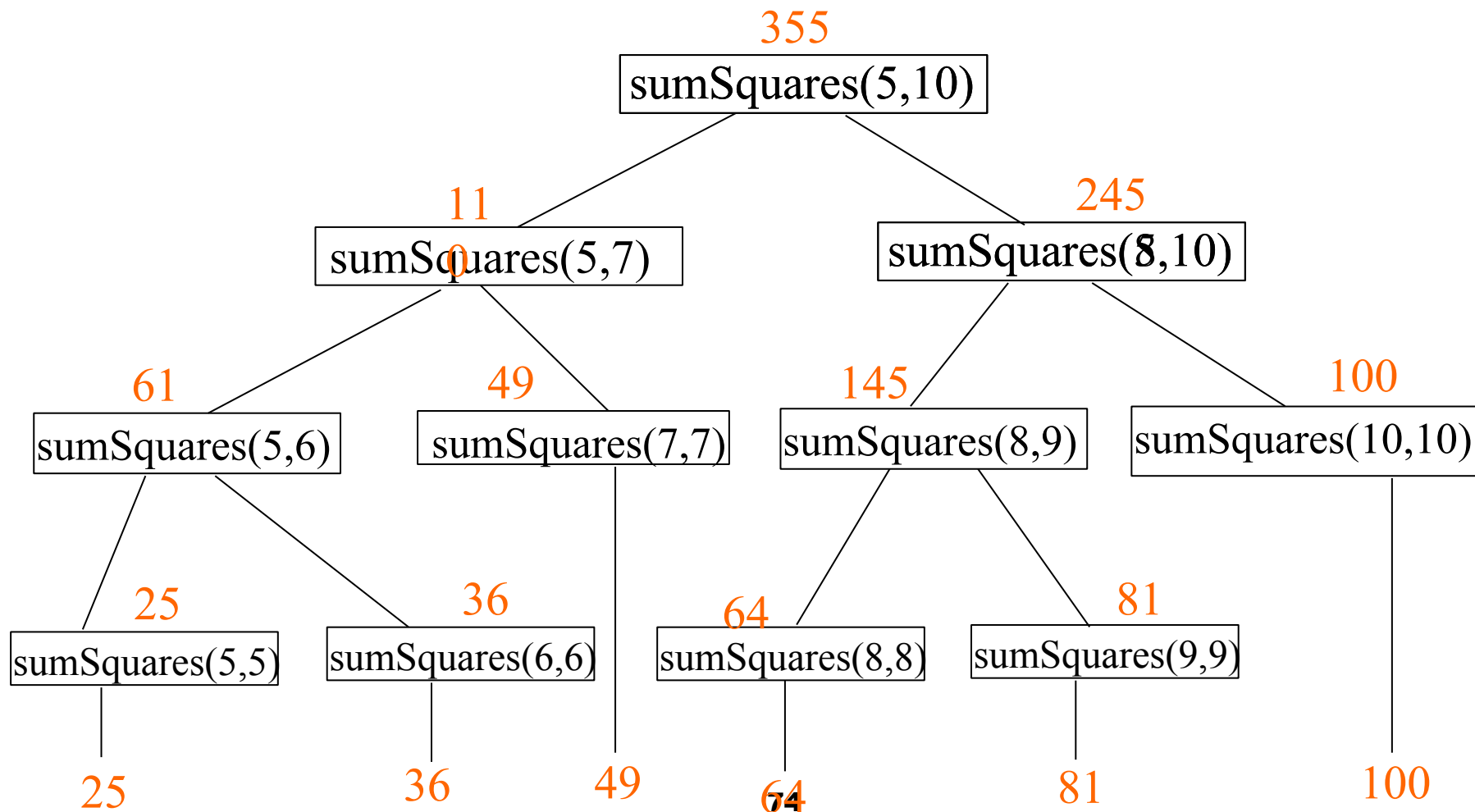


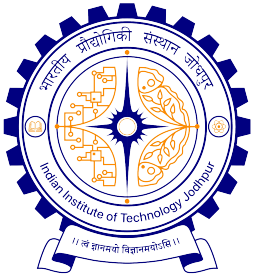
Sum of Squares

```
int sumSquares (int m, int n)
{
    int middle ;
    if (m == n) return m*m;
    else
    {
        middle = (m+n)/2;
        return sumSquares(m,middle)
            +
            sumSquares(middle+1,n) ;
    }
}
```

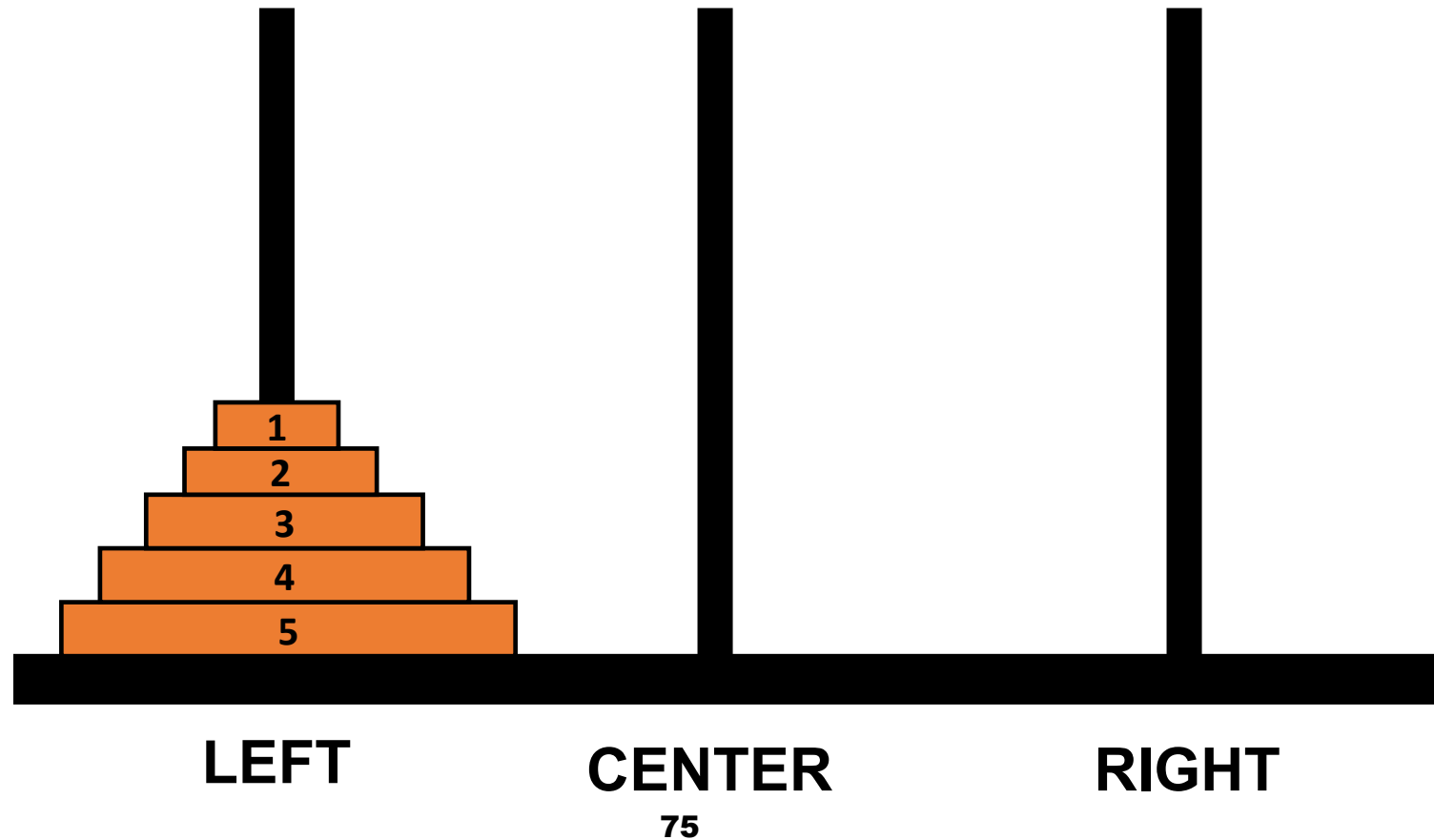


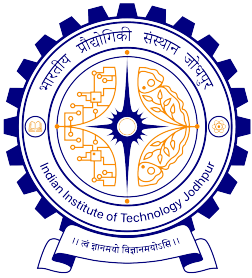
Annotated Call Tree





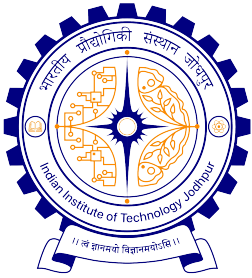
Towers of Hanoi Problem





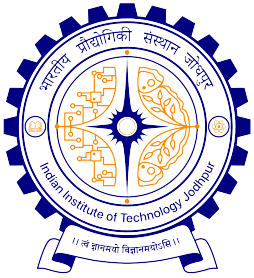
Towers of Hanoi Problem

- Initially all the disks are stacked on the LEFT pole
- Required to transfer all the disks to the RIGHT pole
 - Only one disk can be moved at a time.
 - A larger disk cannot be placed on a smaller disk
- CENTER pole is used for temporary storage of disks



Towers of Hanoi Problem

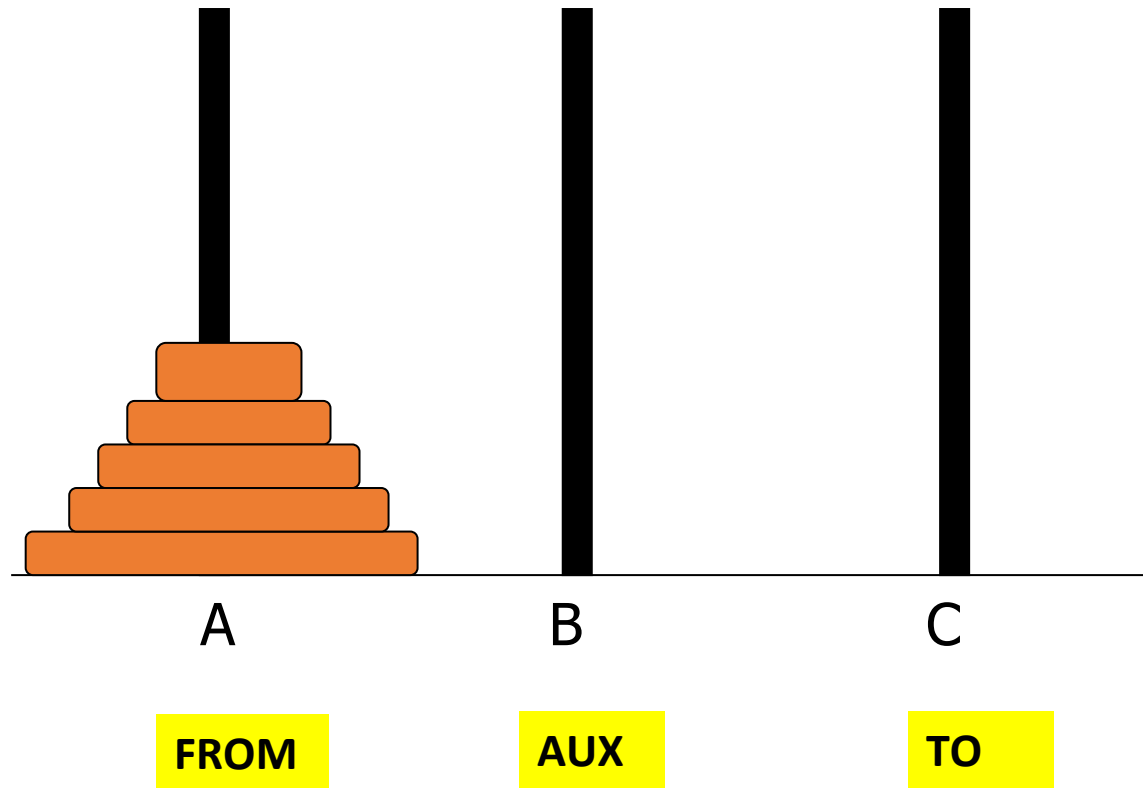
- Recursive statement of the general problem of n disks:
 - Step 1:
 - Move the top $(n-1)$ disks from LEFT to CENTER
 - Step 2:
 - Move the largest disk from LEFT to RIGHT
 - Step 3:
 - Move the $(n-1)$ disks from CENTER to RIGHT

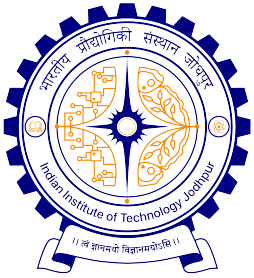


Tower of Hanoi

aim: FROM ? TO via AUX

```
void towers (int n, char from, char to, char aux)
```

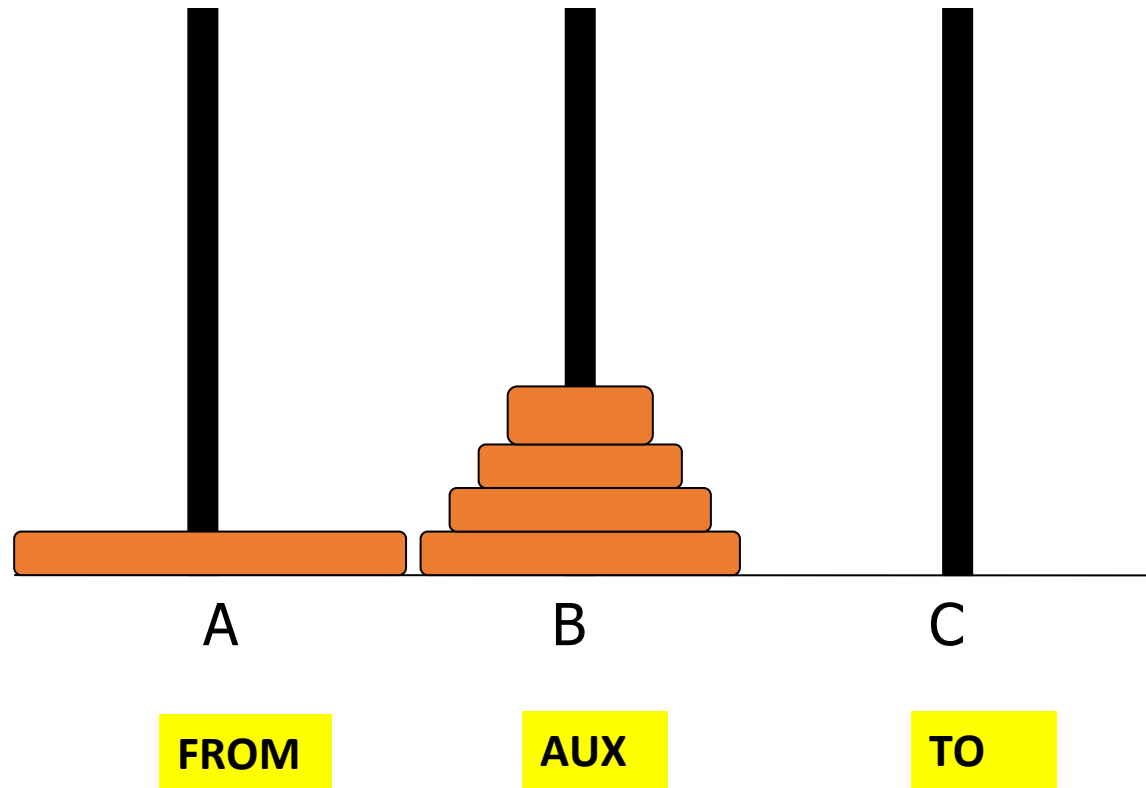


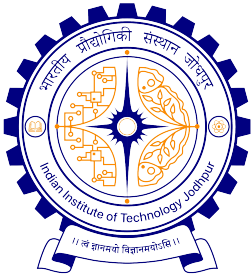


Tower of Hanoi

1. aim: take (n-1) FROM ? AUX

towers (n-1, from, aux, to)

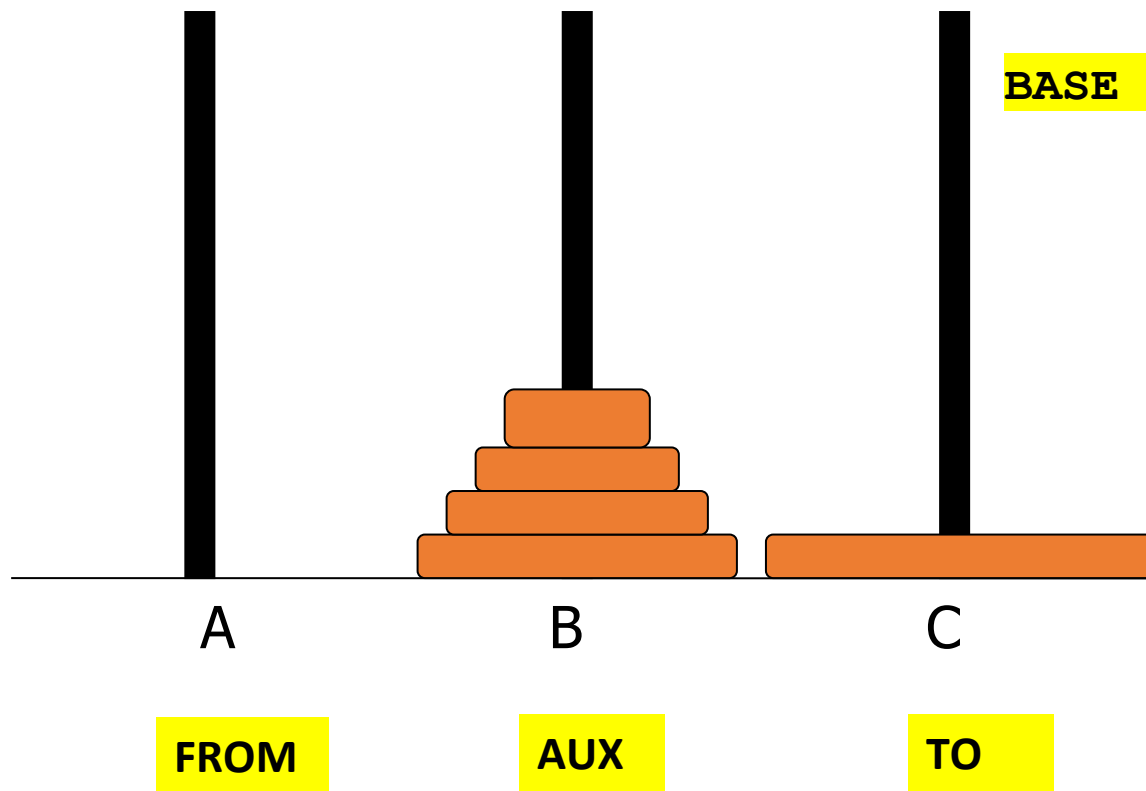


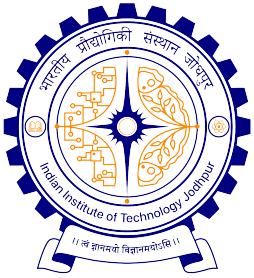


Tower of Hanoi

1. aim: take (n-1) TO **AUX**
and last 1 TO **FROM**

BASE CASE. When $n==1$ Print



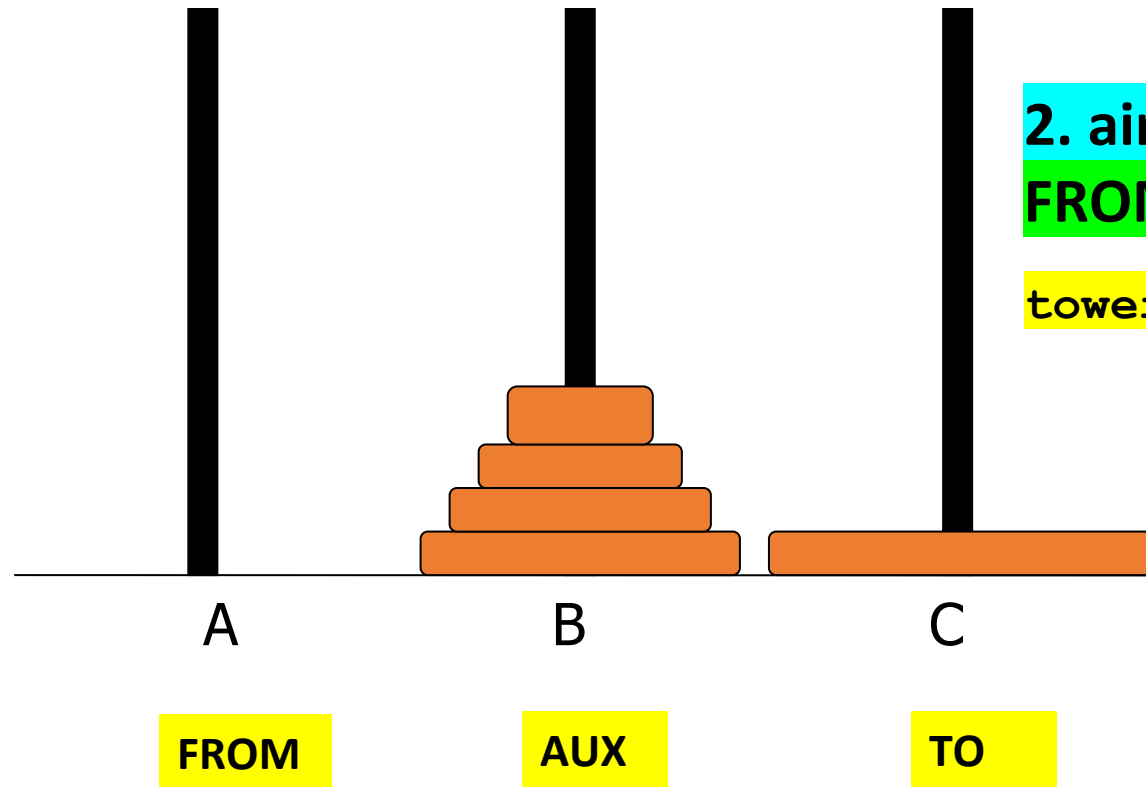


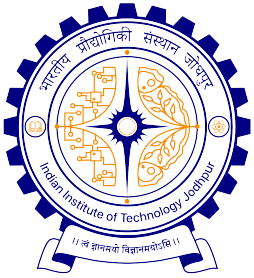
Tower of Hanoi

1. aim: take (n-1) TO $\rightarrow$ AUX
and last 1 TO $\rightarrow$ FROM

2. aim: take (n-1) AUX $\rightarrow$ TO via
FROM

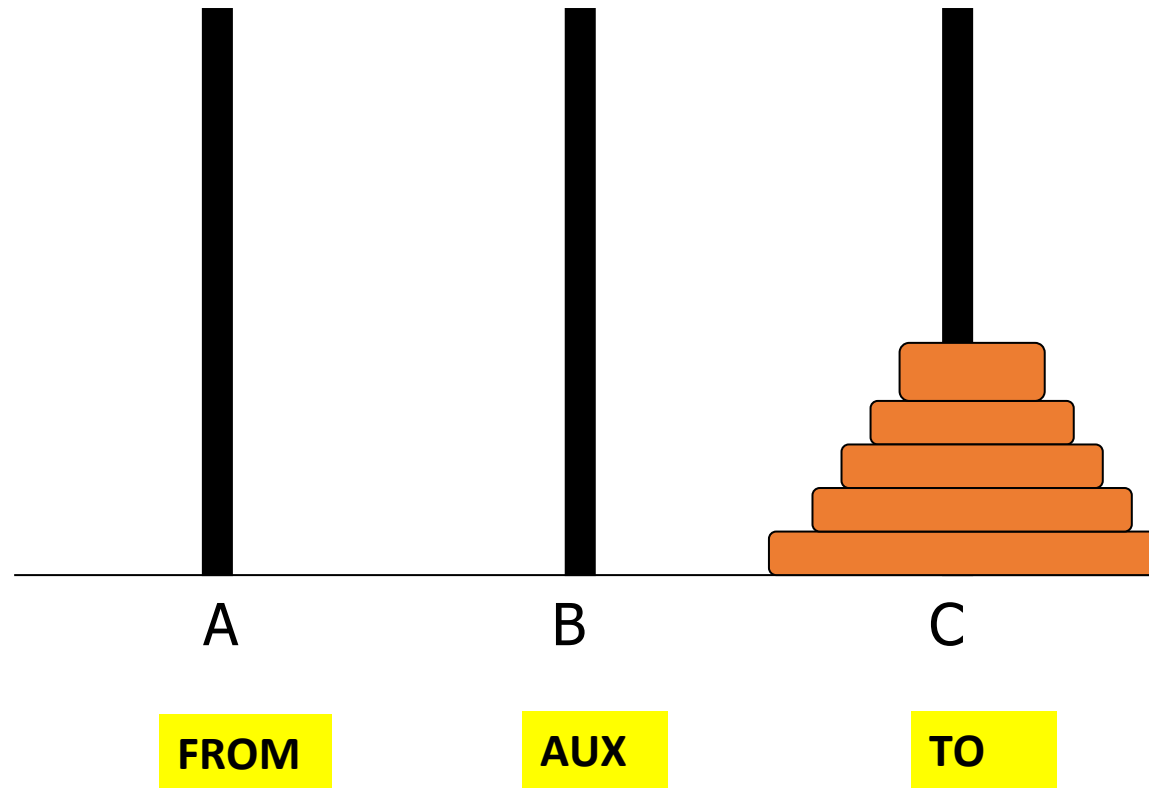
towers (n-1, aux, to, from)

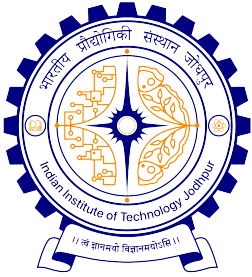




Tower of Hanoi

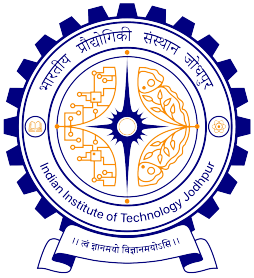
aim achieved!!





Towers of Hanoi function

```
void towers (int n, char from, char to, char aux)
{
    /* Base Condition */
    if (n==1) {
        printf ("Disk 1 : %c  &c \n", from, to) ;
        return ;
    }
    /* Recursive Condition */
    towers (n-1, from, aux, to) ;
    .....
    .....
}
```



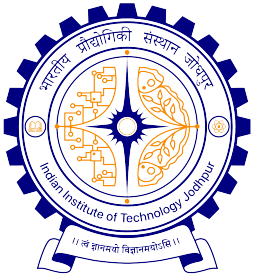
Towers of Hanoi function

```
void towers (int n, char from, char to, char aux)
{
    /* Base Condition */
    if (n==1)    {
        printf ("Disk 1 : %c □ &c \n", from, to) ;
        return ;
    }
    /* Recursive Condition */
    towers (n-1, from, aux, to) ;
    printf ("Disk %d : %c □ %c\n", n, from, to)

    ;

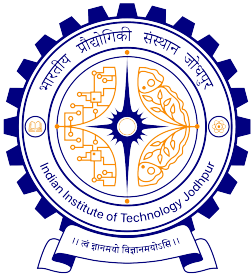
    ..... •

}
```



Towers of Hanoi function

```
void towers (int n, char from, char to, char aux)
{
    /* Base Condition */
    if (n==1)    {
        printf ("Disk 1 : %c □ %c \n", from, to) ;
        return ;
    }
    /* Recursive Condition */
    towers (n-1, from, aux, to) ;
    printf ("Disk %d : %c □ %c\n", n, from, to) ;
    towers (n-1, aux, to, from) ;
}
```



TOH runs

```
void towers(int n, char from, char to, char aux)
{ if (n==1)
  { printf ("Disk 1 : %c -> %c \n", from, to) ;
    return ;
  }

  towers (n-1, from, aux, to) ;
  printf ("Disk %d : %c -> %c\n", n, from, to) ;
  towers (n-1, aux, to, from) ;
}

void main()
{ int n;
  scanf ("%d", &n) ;
  towers (n, 'A' , 'C' , 'B') ;
}
```

Output

3

Disk 1 : A -> C

Disk 2 : A -> B

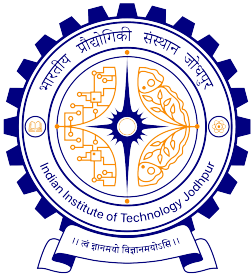
Disk 1 : C -> B

Disk 3 : A -> C

Disk 1 : B -> A

Disk 2 : B -> C

Disk 1 : A -> C



More TOH runs

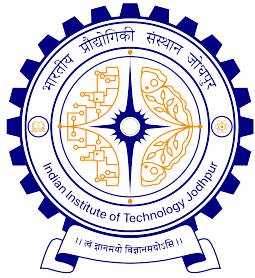
```
void towers(int n, char from, char to, char aux)
{ if (n==1)
  { printf ("Disk 1 : %c -> %c \n", from, to) ;
    return ;
  }
  towers (n-1, from, aux, to) ;
  printf ("Disk %d : %c -> %c\n", n, from, to) ;
  towers (n-1, aux, to, from) ;
}

void main()
{ int n;
  scanf ("%d", &n) ;
  towers (n, 'A' , 'C' , 'B') ;
}
```

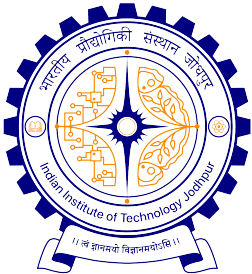
Output

4

Disk 1 : A -> B
Disk 2 : A -> C
Disk 1 : B -> C
Disk 3 : A -> B
Disk 1 : C -> A
Disk 2 : C -> B
Disk 1 : A -> B
Disk 4 : A -> C
Disk 1 : B -> C
Disk 2 : B -> A
Disk 1 : C -> A
Disk 3 : B -> C
Disk 1 : A -> B
Disk 2 : A -> C
Disk 1 : B -> C



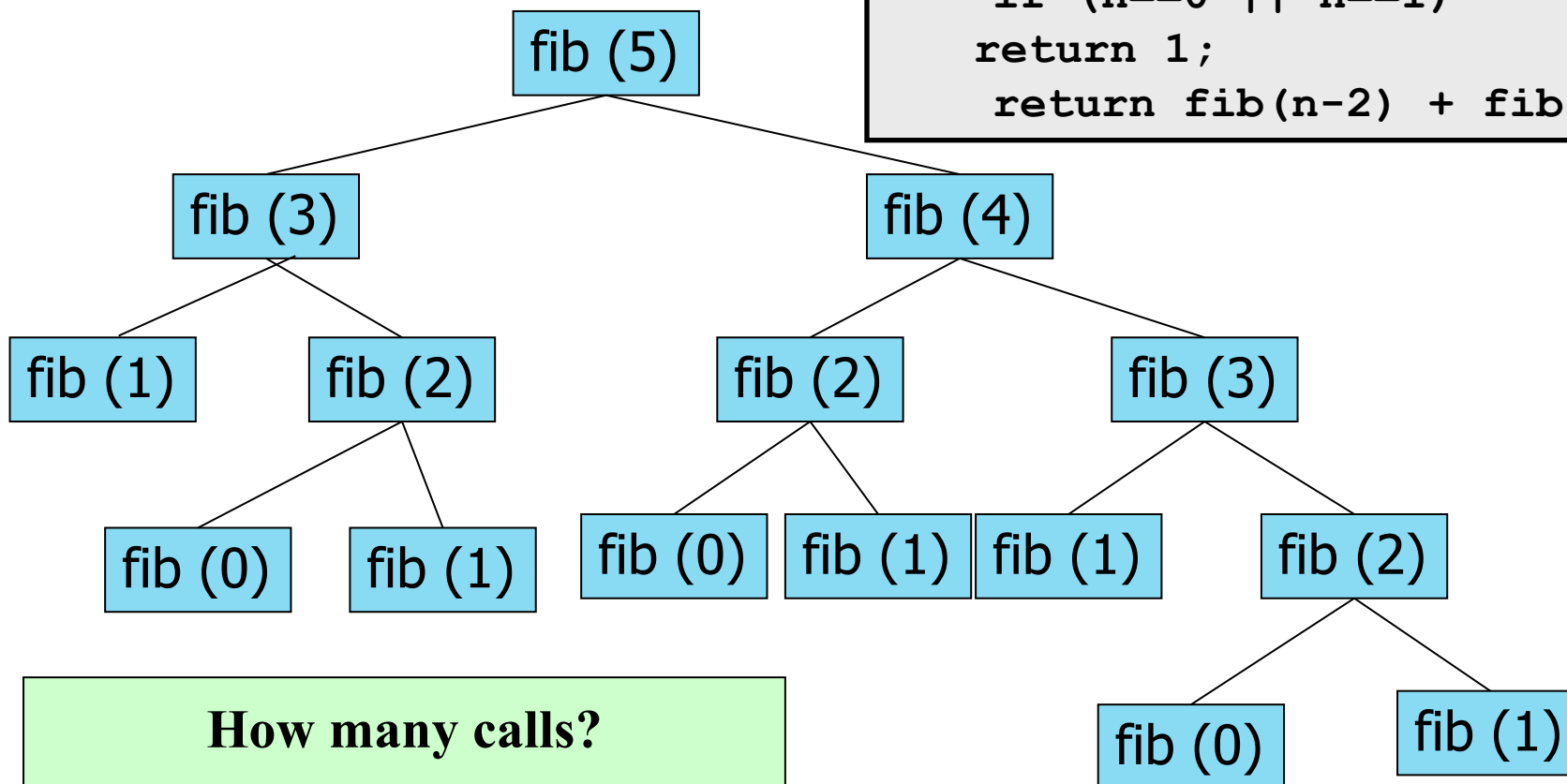
How many times the function `TOH()` will be called to move n disks?



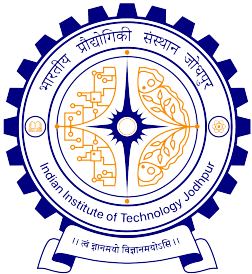
Relook at recursive Fibonacci:

Not efficient !! Same sub-problem solved many times.

```
int fib (int n) {  
    if (n==0 || n==1)  
        return 1;  
    return fib(n-2) + fib(n-1) ;  
}
```



How many calls?
How many additions?

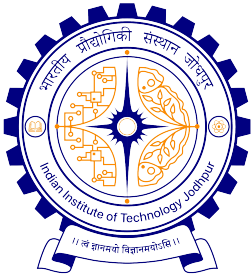


Iterative Fib

Much Less Computation here!
(How many additions?)

```
int fib( int n)
{ int i=2, res=1, m1=1, m2=1;
  if (n ==0 || n ==1) return res;
  for ( ; i<=n; i++)
  { res = m1 + m2;
    m2 = m1;
    m1 = res;
  }
  return res;
}

void main()
{ int n;
  scanf("%d", &n);
  printf(" Fib(%d) = %d \n", n, fib(n));
}
```



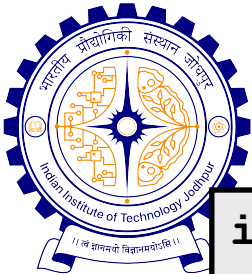
An efficient recursive Fib

```
int Fib ( int, int, int, int);

void main()
{
    int n;
    scanf("%d", &n);
    if (n == 0 || n ==1)
        printf("F(%d) = %d \n", n, 1);
    else
        printf("F(%d) = %d \n", n, Fib(1,1,n,2));
}
```

```
int Fib(int m1, int m2, int n, int i)
{
    int res;
    if (n == i)
        res = m1+ m2;
    else
        res = Fib(m1+m2, m1, n, i+1);
    return res;
}
```

Much Less Computation here!
(How many calls/additions?)



Run

```
int Fib ( int, int, int, int);
void main()
{ int n;
  scanf("%d", &n);
  if (n == 0 || n ==1)  printf("F(%d) = %d \n", n, 1);
  else printf("F(%d) = %d \n", n, Fib(1,1,n,2));
}
int Fib(int m1, int m2, int n, int i)
{ int res;
  printf("F: m1=%d, m2=%d, n=%d, i=%d\n",
                                                m1,m2,n,i) ;

  if (n == i)
    res = m1+ m2;
  else
    res = Fib(m1+m2, m1, n, i+1);
  return res;
}
```

Output

\$./a.out

3

F: m1=1, m2=1, n=3, i=2

F: m1=2, m2=1, n=3, i=3

F(3) = 3

\$./a.out

5

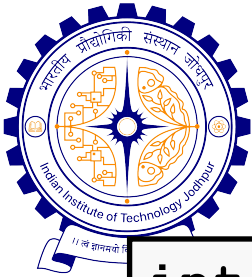
F: m1=1, m2=1, n=5, i=2

F: m1=2, m2=1, n=5, i=3

F: m1=3, m2=2, n=5, i=4

F: m1=5, m2=3, n=5, i=5

F(5) = 8



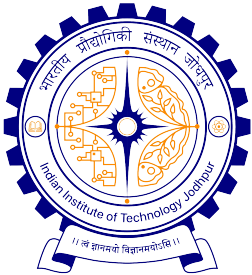
Static Variables

```
int Fib (int, int);

void main()
{
    int n;
    scanf("%d", &n);
    if (n == 0 || n ==1)
        printf("F(%d) = %d \n", n, 1);
    else
        printf("F(%d) = %d \n", n,
Fib(n,2));
}
```

```
int Fib(int n, int i)
{
    static int m1, m2;
    int res, temp;
    if (i==2) {m1 =1; m2=1;}
    if (n == i)   res = m1+ m2;
    else
    {
        temp = m1;
        m1 = m1+m2;
        m2 = temp;
        res = Fib(n, i+1);
    }
    return res;
}
```

Static variables remain in existence rather than coming and going
each time a function is activated



Static Variables: See the addresses!

Output

```
int Fib(int n, int i)
{
    static int m1, m2;
    int res, temp;
    if (i==2) {m1 =1; m2=1;}
    printf("F: m1=%d, m2=%d, n=%d, i=%d\n", m1,m2,n,i) ;
    printf("F: &m1=%u, &m2=%u\n", &m1,&m2) ;
    printf("F: &res=%u, &temp=%u\n", &res,&temp) ;
    if (n == i)  res = m1+ m2;
    else {  temp = m1;  m1 = m1+m2;
           m2 = temp;
           res = Fib(n, i+1);      }
    return res;
}
```

5

F: m1=1, m2=1, n=5, i=2

F: &m1=134518656, &m2=134518660

F: &res=3221224516, &temp=3221224512

F: m1=2, m2=1, n=5, i=3

F: &m1=134518656, &m2=134518660

F: &res=3221224468, &temp=3221224464

F: m1=3, m2=2, n=5, i=4

F: &m1=134518656, &m2=134518660

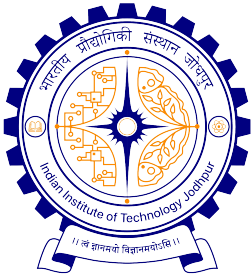
F: &res=3221224420, &temp=3221224416

F: m1=5, m2=3, n=5, i=5

F: &m1=134518656, &m2=134518660

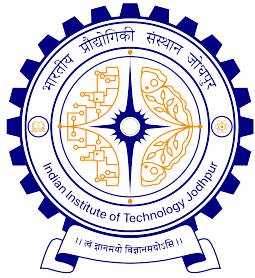
F: &res=3221224372, &temp=3221224368

F(5) = 8



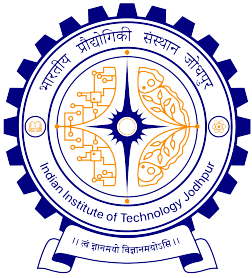
Recursion vs. Iteration

- Repetition
 - Iteration: explicit loop
 - Recursion: repeated function calls
- Termination
 - Iteration: loop condition fails
 - Recursion: base case recognized
- Both can have infinite loops
- Balance
 - Choice between performance (iteration) and good software engineering (recursion).



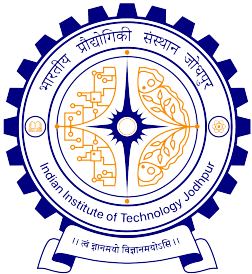
More on Recursion

- Every recursive program can also be written without recursion
- Recursion is used for programming convenience, not for performance enhancement
- Sometimes, if the function being computed has a nice recurrence form, then a recursive code may be more readable



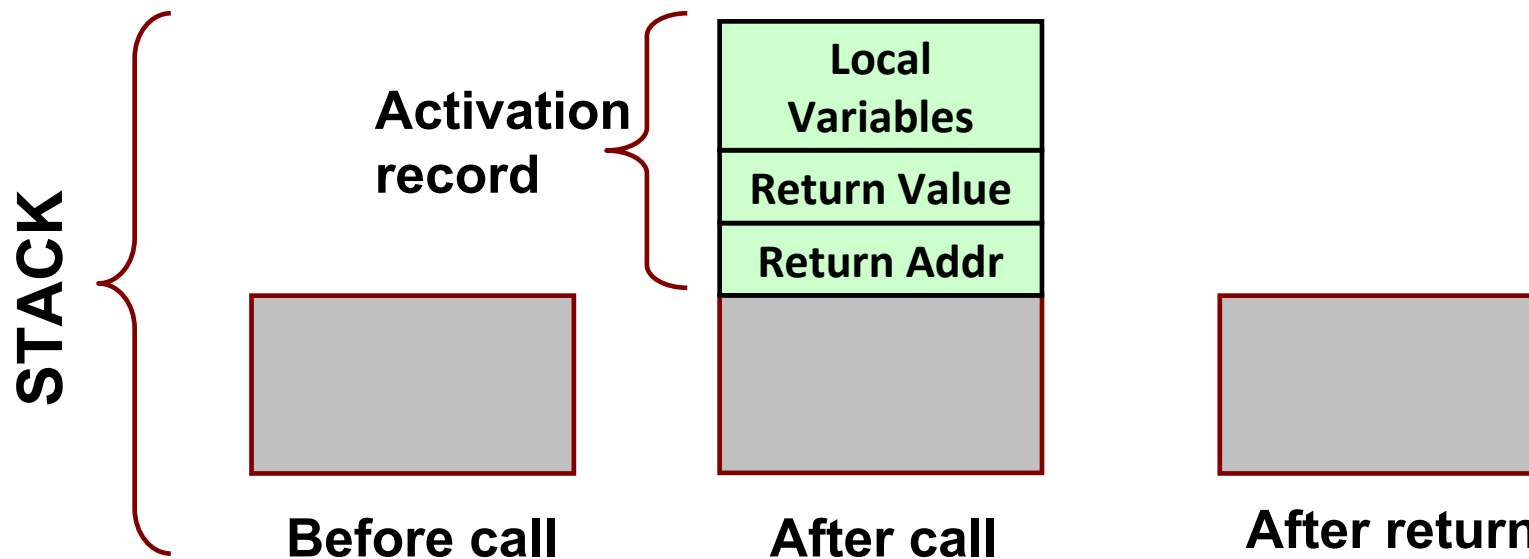
How are function calls implemented?

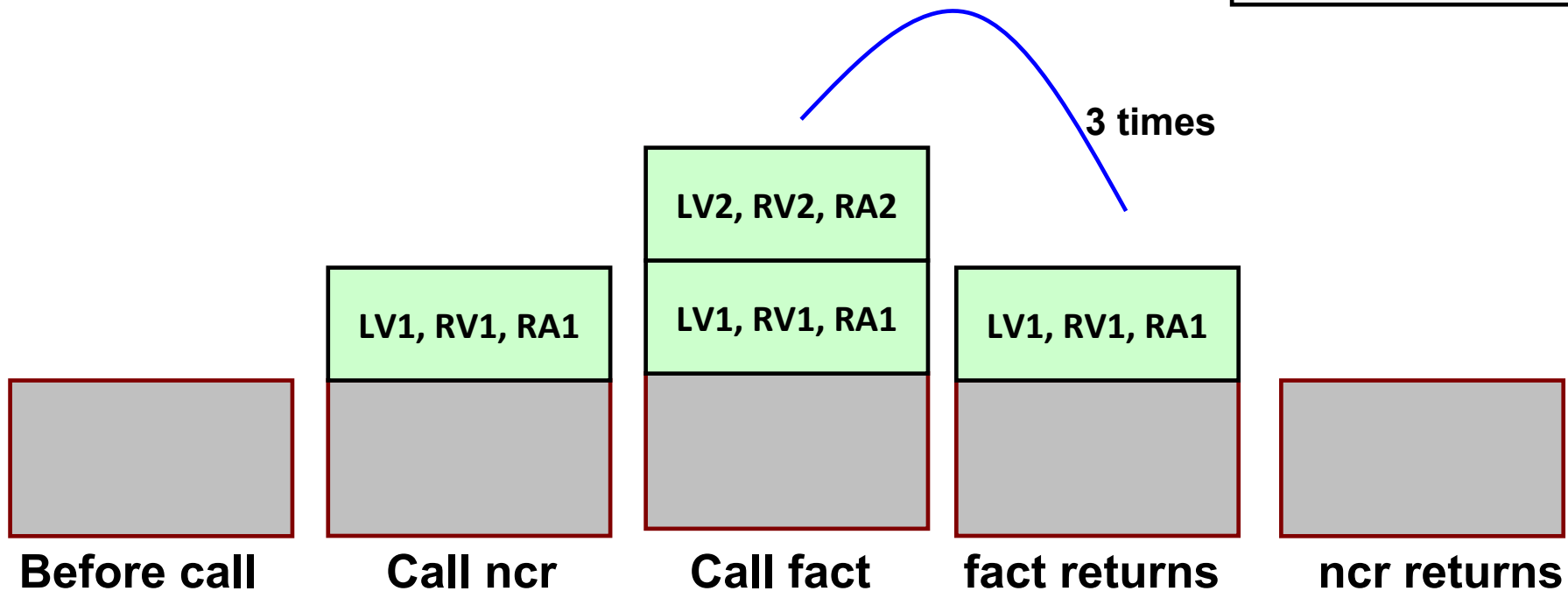
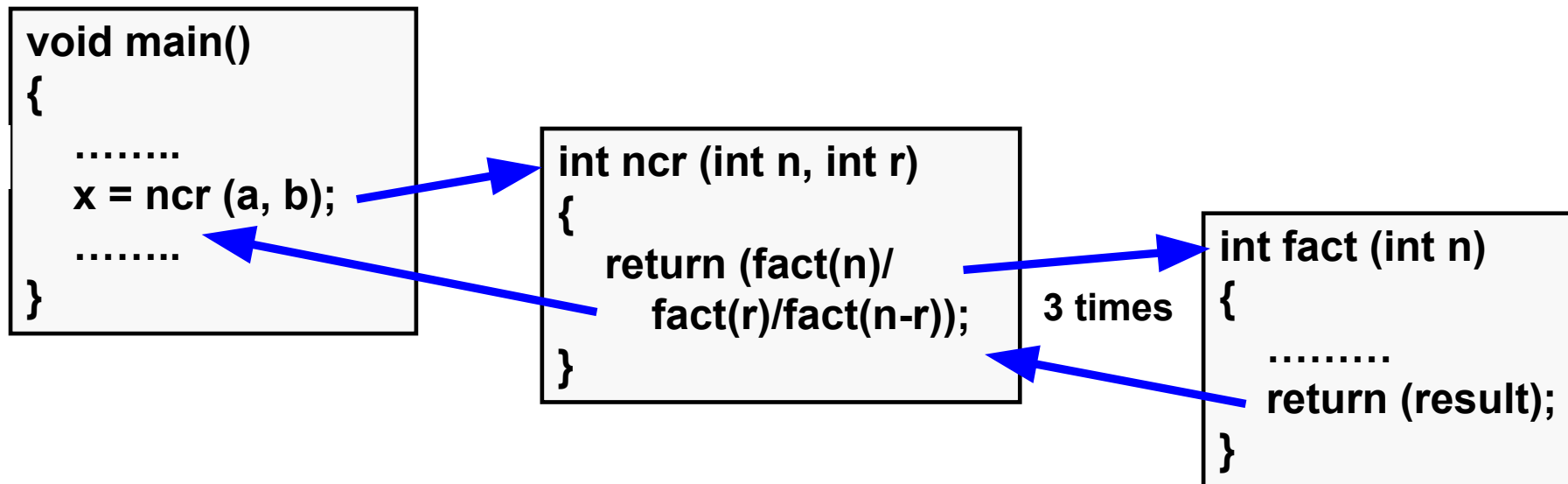
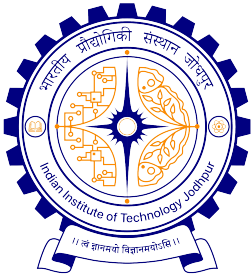
- The following applies in general, with minor variations that are implementation dependent
 - The system maintains a stack in memory
 - Stack is a last-in first-out structure
 - Two operations on stack, push and pop
- Whenever there is a function call, the activation record gets pushed into the stack
 - Activation record consists of the return address in the calling program, the return value from the function, and the local variables inside the function

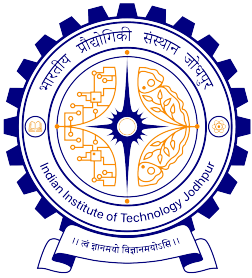


```
void main()  
{  
    .....  
    x = gcd (a, b);  
    .....  
}
```

```
int gcd (int x, int y)  
{  
    .....  
    .....  
    return (result);  
}
```

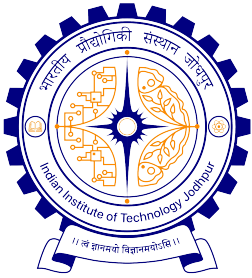






What happens for recursive calls?

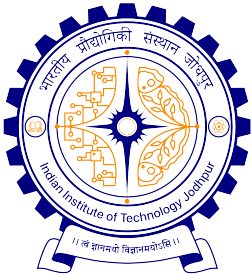
- What we have seen
 - Activation record gets pushed into the stack when a function call is made
 - Activation record is popped off the stack when the function returns
- In recursion, a function calls itself
 - Several function calls going on, with none of the function calls returning back
 - Activation records are pushed onto the stack continuously
 - Large stack space required



What happens for recursive calls?

- Activation records keep popping off, when the termination condition of recursion is reached
- We shall illustrate the process by an example of computing factorial
- Activation record looks like:

Local Variables
Return Value
Return Addr



Example:: main() calls fact(3)

```
void main()
{
    int n;
    n = 3;
    printf ("%d \n", fact(n) );
}
```

```
int fact (n)
int n;
{
    if (n == 0)
        return (1);
    else
        return (n * fact(n-1));
}
```

